

Correction du TD 1 – Introduction à la Compilation

Partie A – Questions de cours (10 points)

Q1. (1 pt) Définition du processus de compilation

La compilation est un processus de traduction qui transforme un programme écrit dans un langage source (haut niveau) en un programme équivalent dans un langage cible (bas niveau, souvent en code machine). Ce processus comprend généralement plusieurs phases d'analyse et de synthèse.

Q2. (2 pts) Six phases principales de compilation

1. **Analyse lexicale** : Découpe le code source en tokens (unités lexicales).
2. **Analyse syntaxique** : Vérifie la structure grammaticale et construit un arbre syntaxique.
3. **Analyse sémantique** : Vérifie la cohérence des types et des déclarations.
4. **Génération de code intermédiaire** : Produit une représentation intermédiaire indépendante de la machine.
5. **Optimisation** : Améliore le code intermédiaire pour des performances accrues.
6. **Génération de code machine** : Traduit le code optimisé en langage cible.

Q3. (1 pt) Arbre syntaxique concret vs AST

- **Arbre syntaxique concret** : Représentation détaillée incluant tous les éléments grammaticaux (parenthèses, opérateurs, etc.).
- **AST (Arbre de Syntaxe Abstraite)** : Simplifié, ne retient que les éléments essentiels à la sémantique du programme (opérations, variables, etc.).

Q4. (1 pt) Frontend vs Backend

- **Frontend** : Gère les phases d'analyse (lexicale, syntaxique, sémantique) et dépend du langage source.
- **Backend** : Gère la génération et l'optimisation du code cible, dépendant de l'architecture machine.

Q5. (1 pt) Utilité de la table des symboles

La table des symboles stocke les informations sur les identifiants (variables, fonctions, etc.) comme leur type, portée, et adresse mémoire. Elle est cruciale pour la vérification sémantique et la génération de code.

Q6. (1 pt) Erreurs détectées durant la compilation

- **Erreur lexicale** : Caractère invalide (détectée lors de l'analyse lexicale).
- **Erreur syntaxique** : Structure incorrecte (détectée lors de l'analyse syntaxique).

Q7. (1 pt) Rôle du chargeur et de l'éditeur de liens

- **Chargeur (loader)** : Charge le programme compilé en mémoire pour exécution.
- **Éditeur de liens (linker)** : Combine plusieurs modules objet en un seul exécutable, résolvant les références croisées.

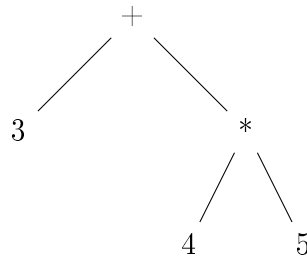
Q8. (2 pts) Outils pour la construction de compilateurs

- **Lex/Yacc** : Génèrent des analyseurs lexicaux (Lex) et syntaxiques (Yacc).
- **LLVM** : Infrastructure pour l'optimisation et la génération de code multi-plateforme.
- **ANTLR** : Outil puissant pour générer des analyseurs syntaxiques à partir de grammaires.

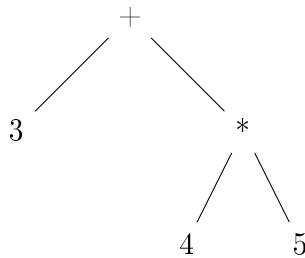
Partie B – Questions d'analyse et réflexion (6 points)

Q9. (2 pts) Exemple d'expression et arbres

Expression : $3 + 4 * 5$



— **Arbre syntaxique concret** :



— **AST** :

Q10. (2 pts) Importance du code intermédiaire

Le code intermédiaire permet une séparation claire entre les phases d'analyse et de synthèse, facilitant la portabilité et les optimisations. Générer directement le code machine limiterait la flexibilité et rendrait le compilateur dépendant d'une architecture spécifique.

Q11. (2 pts) Risques de l'optimisation

L'optimisation peut introduire des bugs si elle modifie involontairement la sémantique du programme. Exemple : Supprimer une boucle vide qui servait à une temporisation.

Partie C – Mini recherche (4 points)

Q12. (2 pts) Contribution de Aho et al.

L'ouvrage de Aho, Sethi, et Ullman ("Compilateurs : Principes, Techniques et Outils") est une référence majeure. Il formalise les concepts clés de la compilation (analyse LR, génération de code) et reste un fondement académique et industriel.

Q13. (2 pts) Caractéristiques d'un compilateur moderne

LLVM :

- Infrastructure modulaire pour la compilation, l'optimisation, et la génération de code.
- Utilisée par Clang (frontend pour C/C++) et supportant multiples langages et cibles.
- Optimisations agressives et représentation intermédiaire (IR) flexible.